TP Arbres de Decision

Licence Informatique - 3ème année

Le but de ce TP est d'implementer l'algorithme de création d'arbre de décision vu en cours. Vous disposez pour cela d'un peu de code de départ permettant de charger la base de donnée à partir de laquelle votre arbre apprendra, une classe DataPoint pour structurer les points de donnée, et le début de la classe pour l'arbre. Notez que les questions et les structures peuvent être légèrement altérées, si cela vous permet d'avoir des objets que vous vous sentez plus capables de manipuler ou de mettre en relation avec les autres.

Par soucis de simplification, on ne considèrera une structure d'arbre que pour des problèmes de classification avec deux classes, 0 et 1, et des exemples dont les attributs ont des valeurs réelles, à séparer via une question de type $attribut < s$ ou non.

Pour commencer, chargez les données avec la fonction **load_data** fournie. Notez que **load_data** renvoie une liste d'instances de la classe DataPoint, qui contiennent un attribut x, sous forme d'un dictionnaire des entrées, et un attribut y, correspondant à ce que l'on veut prédire.

1. Creez une fonction **proba_empirique**, prenant d une liste de DataPoint, et renvoyant la liste des p_i, où p_i est la proportion d'éléments de d de classe i.

2. Creez une fonction **question_inf**, prenant un x de DataPoint, le nom d'un attribut a et une valeur s, et renvoyant vrai si $x[a] < s$, faux sinon. Prenez soin d'utiliser uniquement cette fonction si vous souhaitez faire une comparaison sur un attribut, afin d'éviter d'eventuelles erreurs de signe ou de format. Par la suite on appellera (a, s) la question de si $x[a] < s$ ou non.

3. Ajoutez une fonction **split**, prenant une liste de DataPoint d, le nom d'un attribut a et une valeur s, et renvoyant deux listes d_1 et d_2 correspondant à la partition de d suivant la question (a, s). On rappelle que la liste vide est "[]" et l'on peut ajouter des points à la fin d'une liste via la méthode **append**.

4. Ajoutez une fonction **list_separ_attributs**, prenant une liste de DataPoint d et un nom d'attribut a, et renvoyant une liste de questions de type (a, s) pour tous les s pertinents. Entre deux valeurs, vous privilegerez la moyenne des deux. Vous pourrez utiliser pour cela la fonction **set**, qui peut transformer une liste en un ensemble sans doublon, puis la fonction **list** qui pourra retransformer l'ensemble en liste. La méthode de liste **sort** permettra également d'ordonner la liste.

5. Déduisez en la fonction **liste_questions**, qui étant donnée une liste de DataPoint d, renvoie la liste des questions pertinentes de forme (a, s) à poser sur les éléments de d.

6. Créez une fonction **entropie**, prenant une liste de DataPoint et renvoyant l'entropie de cette liste. Pour rappel, si p_i est la proportion d'élements de d de classe i, alors l'entropie H est définie par

$$H(d) := -\sum_i p_i \log(p_i) \ .$$

   Vous ferez attention au cas où p_i est égal à 0, où $\log(p_i)$ n'est pas défini, mais où par convention on évaluera $p_i \log(p_i)$ à 0.

7. En déduire une fonction **gain_entropie**, prenant une liste de DataPoint d, une question (a, s) et renvoyant le gain d'entropie apporté par la question (a, s). Si l'on note d_1 et d_2 la partition de d par (a, s) et r_1 et r_2 leurs proportions respectives de points de d, pour rappel, le gain d'entropie est

$$G(d, (a, s)) := H(d) - r_1 H(d_1) - r_2 H(d_2) \ .$$

8. En déduire une fonction **best_split** qui renvoie une meilleure question (a, s) possible, en terme de maximisation du gain d'entropie.

9. Ajoutez une méthode de classe Noeud, **grow**, prenant une liste de DataPoint d. La méthode calculera l'entropie de d, la question (a, s) apportant le meilleur split de d en une partition d_1 et d_2. Si l'entropie est positive, que la profondeur maximale de l'arbre n'est pas atteinte, et que d_1 et d_2 contiennent chacun au moins un élément, alors la méthode enregistrera la question, créera deux nouveaux Noeuds enfants sur lesquels elle se relancera pour d_1 et d_2. La question (a, s) sera mise dans un attribut **question**. Autrement, le noeud correspond à une feuille. La méthode donnera un attribut **proba** à l'objet qui prendra pour valeur ce que renvoie **proba_empirique**.

10. Ajoutez une méthode **prediction**, prenant un x et renvoyant la probabilité prédite pour x par le Noeud.

11. Ajoutez un attribut **hauteur** correspondant à la hauteur du Noeud dans l'arbre.

12. Créez une fonction **precision**, prenant un Noeud et une liste de DataPoint, et renvoyant le pourcentage de DataPoint correctement prédits.

13. Découpez l'ensemble des points de données en deux sous-ensemble : 80% des points iront dans une base d'apprentissage, le reste dans une base de test. En fonction de la profondeur maximale, tracez l'évolution de la précision sur la base d'apprentissage et la base de test. Que constatez vous ?

14. Ajoutez une méthode **elagage** à la classe Noeud, prenant un coefficient α, et à partir de la fin de l'arbre, retirant les noeuds jusqu'à trouver des noeuds ayant eu un gain d'entropie supérieur à α.

15. Tracez l'évolution de la précision sur la base d'apprentissage et sur la base de test en fonction de α.

Suite bonus : Forêts Aléatoires

16. Créez une méthode **grow_partial** de classe Noeud, prenant une liste de DataPoint d et un entier k et selectionnant au hasard k des attributs de x qui seront autorisés pour poser une question sur x. La méthode selectionnera parmi ces k attributs lequel donne lieu au gain maximal, et continuera sur les enfants pour k autres attributs aléatoires jusqu'à son arrêt. Lors de vos tests, donnez à k une valeur égale à la racine carrée du nombre d'attributs de x.

17. Créez une classe ForetAleatoire, qui contiendra un ensemble de Noeuds.

18. Donnez à la classe ForetAleatoire une méthode **prediction**, prenant un x et renvoyant la probabilité moyenne prédite par les arbres de la ForetAleatoire.

19. Créez une fonction **bootstrap**, prenant une liste de DataPoint d et un entier n, et renvoyant une liste de DataPoint comprenant n points, tirés de d par un tirage aléatoire uniforme avec remise.

20. Equipez votre classe ForetAleatoire d'une méthode **grow**, prenant en entrée une liste d de DataPoint et un entier k. La méthode grandira k Noeuds différents via leurs méthodes **grow_partial**, en donnant à chacun des ces arbres une liste de DataPoint pour leur apprentissage différente, obtenue via la fonction **bootstrap** pour n égal aux nombres d'éléments de d.

21. Faites grandir une ForetAleatoire sur votre base d'apprentissage, et tracez l'évolution de la précision sur la base d'apprentissage et la base de test en fonction du nombre d'arbres de la ForetAleatoire utilisés pour la prédiction.